

23 — SQLite Vertiefung

WHERE, ORDER BY, JOIN, UPDATE/DELETE, GROUP BY & Aggregate

HTW Berlin – Wirtschaftsingenieurwesen

SoSe 2026

Agenda

1. WHERE feiner: AND/OR, BETWEEN, IN, LIKE, IS NULL
2. ORDER BY – Ergebnisse sortieren
3. JOIN – Tabellen verbinden (INNER, LEFT)
4. UPDATE & DELETE – ändern und löschen (vorsichtig!)
5. Aggregatfunktionen + GROUP BY + HAVING – Auswertungen
6. Der Bauplan einer Abfrage – in welcher Reihenfolge das passiert

Lernziel: Mit SQL gezielt filtern, sortieren, Tabellen verknüpfen, Daten ändern und Kennzahlen berechnen.

Wie wir heute arbeiten: Konzept → *Live-Demo* (Veggie-Soles-Shop, In-Memory-DB) → *Sofort ausprobieren* (Notebook 23, Chinook-Datenbank).

Von „alles holen“ zu „genau das brauchen“

SELECT * FROM produkt gibt *alle* Spalten und *alle* Zeilen zurück. In der Praxis will man:

- nur bestimmte Zeilen (WHERE), in einer Reihenfolge (ORDER BY)
- Daten aus mehreren Tabellen (JOIN)
- Daten auch *ändern* (UPDATE, DELETE)
- *zusammengefasst*: Anzahl, Summe, Durchschnitt (GROUP BY + Aggregate)

```
SELECT kunde.name, SUM(bestellung.betrag) AS umsatz
FROM kunde JOIN bestellung
ON kunde.kunde_id = bestellung.kunde_id
GROUP BY kunde.kunde_id
HAVING umsatz > 200 ORDER BY umsatz DESC;
```

WHERE - gezielt filtern

```
SELECT name, preis FROM produkt
WHERE (kategorie = 'Low-Cut'
      OR kategorie = 'High-Cut')
      AND preis < 100;
```

Werkzeug	Beispiel
Vergleiche / Logik	= != < > · ... AND ... · ... OR ... (Klammern!)
Bereich (inklusiv)	preis BETWEEN 90 AND 120
Liste / Textmuster	kategorie IN ('Boot', 'High-Cut') · name LIKE '%er%'
fehlender Wert	kategorie IS NULL - nicht = NULL!

► **Live-Demo** - 01_where_und_order_by.py

✎ **Sofort ausprobieren** - Notebook 23, Übungen 1.1 & 1.2: Tracks > 300.000 ms; Rock-/Metal-Tracks für 0.99 (mit Klammern!).

ORDER BY - sortieren

-- *günstigste zuerst:*

```
SELECT name, preis FROM produkt ORDER BY preis ASC;
```

-- *nach Kategorie, dann Preis absteigend:*

```
SELECT kategorie, name, preis  
FROM produkt ORDER BY kategorie, preis DESC;
```

- ASC = aufsteigend (Standard, kann weg) · DESC = absteigend
- mehrere Spalten: die **erste** hat Vorrang, die zweite entscheidet bei Gleichstand
- ohne ORDER BY ist die Reihenfolge **nicht garantiert**

 **Sofort ausprobieren** – Notebook 23, Übung 2.1: die 10 günstigsten Tracks (Name + Preis, nach Preis aufsteigend).

JOIN - Tabellen verbinden

JOIN als Mengenoperation: welche Zeilen landen im Ergebnis?

INNER JOIN: Treffer in beiden



FROM kunde JOIN bestellung ON ...

LEFT JOIN: alle links



FROM kunde LEFT JOIN bestellung ON ...

Orange = diese Zeilen stehen im Ergebnis · bei LEFT JOIN bekommen Kund:innen ohne Bestellung NULL in den bestellung-Spalten.

- INNER JOIN – nur Treffer in **beiden** Tabellen · LEFT JOIN – **alle** links, fehlt rechts → NULL · verbunden über ON (FK = PK)

INNER vs. LEFT – konkret

Derselbe JOIN, konkret: kunde + bestellung

kunde	
kunde_id	name
1	Anna
2	Max
3	Lena

bestellung		
bestell_id	kunde_id	betrag
10	1	89.95
11	1	109.00
12	2	135.50

Lena (3) hat keine Bestellung.

INNER JOIN

INNER JOIN — nur passende Paare

name	bestell_id	betrag
Anna	10	89.95
Anna	11	109.00
Max	12	135.50

LEFT JOIN

LEFT JOIN — alle Kund:innen

name	bestell_id	betrag
Anna	10	89.95
Anna	11	109.00
Max	12	135.50
Lena	NULL	NULL

INNER JOIN "vergisst" Lena — sie hat keinen Treffer. LEFT JOIN behält sie und füllt die fehlenden Werte mit NULL.

INNER JOIN „vergisst“ Zeilen ohne Treffer · LEFT JOIN behält sie – so findet man auch *die ohne* (... WHERE rechte.id IS NULL).

► **Live-Demo** - 02_inner_join.py (3-Tabellen-JOIN) · 03_left_join.py (Kund:innen ohne Bestellung)

✎ **Sofort ausprobieren** - Notebook 23, Übungen 3.1 & 3.2: Tracks mit Gennamen (INNER JOIN); Kunden ohne Rechnung (LEFT JOIN + IS NULL).

UPDATE & DELETE - ändern und löschen

```
UPDATE produkt SET preis = 79.90 WHERE name = 'Kork-Slip';  
DELETE FROM produkt WHERE aktiv = 0;
```

- **immer mit WHERE!** Ohne WHERE trifft es *alle* Zeilen - und ist nicht rückgängig zu machen.
- **erst SELECT** mit derselben WHERE-Bedingung - prüfen, was getroffen würde · Backup vor größeren Änderungen
- mehrere Spalten: SET a = ..., b = ... · auch hier Platzhalter ?

► **Live-Demo** - 04_update_delete.py

✎ **Sofort ausprobieren** - Notebook 23, Übung 4.1: Künstler anlegen und per UPDATE umbenennen.

Aggregatfunktionen - aus vielen Zeilen ein Wert

COUNT(*)	SUM(x)	AVG(x)	MIN(x)	MAX(x)
Zeilen zählen	summieren	Durchschnitt	Minimum	Maximum

```
SELECT COUNT(*), AVG(preis), MAX(preis) FROM produkt;
```

- über die ganze Tabelle: das Ergebnis ist **eine** Zeile
- COUNT(*) zählt alle Zeilen, COUNT(spalte) nur die mit Wert (ohne NULL)

 **Sofort ausprobieren** – Notebook 23, Übung 5.1: Gesamtumsatz pro Kunde (JOIN + GROUP BY).

GROUP BY & HAVING – Auswertung je Gruppe

GROUP BY — viele Zeilen werden zu einem Wert pro Gruppe

bestellung (Rohdaten)

kunde · bestell_id · betrag

Anna	10	89.95
Max	12	135.50
Anna	11	109.00
Lena	13	75.00
Anna	14	60.00
Max	15	89.95

GROUP BY
kunde

Gruppe: Anna
10 · 89.95
11 · 109.00
14 · 60.00

Gruppe: Max
12 · 135.50
15 · 89.95

Gruppe: Lena
13 · 75.00

COUNT(*)
SUM(betrag)

Ergebnis: eine Zeile pro Kund:in

kunde	anzahl	summe
Anna	3	218.95
Max	2	225.45
Lena	1	75.00

- GROUP BY g → ein Aggregat **je Gruppe**; HAVING filtert Gruppen *nach*, WHERE *davor*
- jede SELECT-Spalte: im GROUP BY **oder** in einer Aggregatfunktion

► **Live-Demo** - 05_aggregat_group_by.py

✎ **Sofort ausprobieren** - Notebook 23, Übung 5.2: nur Kunden mit Umsatz > 40 (HAVING).

Der Bauplan einer Abfrage

Wie SQL eine Abfrage abarbeitet — und warum die Reihenfolge zählt

► so wird sie ausgeführt



so wird sie geschrieben:

```
SELECT kunde.name, SUM(bestellung.betrag) AS umsatz
FROM kunde JOIN bestellung ON kunde.kunde_id = bestellung.kunde_id
WHERE bestellung.betrag > 50
GROUP BY kunde.kunde_id
HAVING umsatz > 200
ORDER BY umsatz DESC
LIMIT 10;
```

blau = arbeitet auf Zeilen · orange = arbeitet auf Gruppen

Geschrieben wird `SELECT ... FROM ... WHERE ... GROUP BY ... HAVING ... ORDER BY` – *abgearbeitet* in anderer Reihenfolge. Deshalb kann `WHERE` nicht auf ein Aggregat zugreifen – dafür gibt es `HAVING`.

Cheat Card - Notebook 23

Aufgabe	SQL
filtern	WHERE bedingung - AND/OR, BETWEEN, IN, LIKE, IS NULL
sortieren	ORDER BY spalte [ASC\ DESC]
Tabellen verbinden	... JOIN andere ON a.x = andere.x (INNER / LEFT)
ändern / löschen	UPDATE t SET s = w WHERE ... · DELETE FROM t WHERE ...
zusammenfassen	SELECT g, COUNT(*), SUM(x) FROM t GROUP BY g
Gruppen filtern	... GROUP BY g HAVING aggregat > wert

Faustregel: UPDATE/DELETE *nie* ohne WHERE. WHERE filtert Zeilen, HAVING filtert Gruppen.

Ausblick: Datenbankmodule abgeschlossen

Mit den Notebooks **20-23** haben Sie den Bogen geschlagen:

- **20** – was Datenbanken sind, welche Arten es gibt, warum nicht Excel
- **21** – das relationale Modell: Tabellen, Schlüssel, Beziehungen, ACID
- **21b** – Datenmodelle als ER-Diagramm entwerfen
- **22** – SQLite mit Python: anlegen, einfügen, abfragen
- **23** – fortgeschrittene Abfragen: filtern, sortieren, verbinden, ändern, auswerten

Damit können Sie eine kleine Anwendung mit dauerhafter, strukturierter Datenhaltung bauen – vom ER-Entwurf bis zur Auswertungsabfrage.

Heute gelernt

- ✓ WHERE mit AND/OR, BETWEEN, IN, LIKE, IS NULL
- ✓ ORDER BY – ein- und mehrspaltig, ASC/DESC
- ✓ INNER JOIN und LEFT JOIN – inkl. „die ohne Treffer“ finden
- ✓ UPDATE und DELETE – immer mit WHERE
- ✓ Aggregatfunktionen, GROUP BY, HAVING
- ✓ in welcher Reihenfolge SQL eine Abfrage abarbeitet

Zur Vertiefung im Notebook 23:

- „Sofort ausprobieren“-Übungen 1.1–5.2 (sind Sie schon mitgegangen)
- Abschlussübungen Aufg. 1–4: USA-Kunden mit „J“, Alben von „The ...“, Umsatzanalyse nach Land, längste Tracks pro Genre